

Handling Execution Overruns

Hard real-time control systems • CBS/CBShd • Prototype simulation

Core question

How can a hard real-time system stay safe when a task suddenly needs more execution time than expected?

Christian Percival
Electronic Engineering • Hochschule Hamm-Lippstadt

What is an execution overrun?

A timing problem: the task may still compute the correct result, but it uses more CPU time than the system planned for.

Overrun = actual execution time exceeds the normal/reserved budget

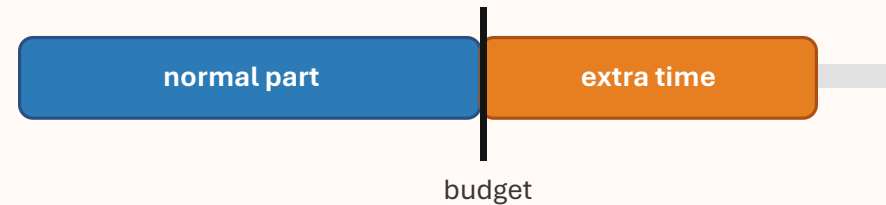
formal idea: $c_i > c_i^n$ and overrun amount = $c_i - c_i^n$

Normal execution



Finishes inside budget → next tasks can run as planned.

Execution overrun



Uses CPU time that was not planned → can delay other jobs.

Different ways to handle execution overruns

Each method makes a different safety/efficiency tradeoff

WCET-based

Reserve enough time for the worst case.

[1][3][4]

Normal-only

Schedule using typical execution time. Good as a fast baseline, but unsafe if hard frames happen in hard real-time.

[1]

Fallback / degrade

Use previous result, skip frame, or lower quality. Useful when approximate output is acceptable and safe.

[1][3]

CBS / CBShd

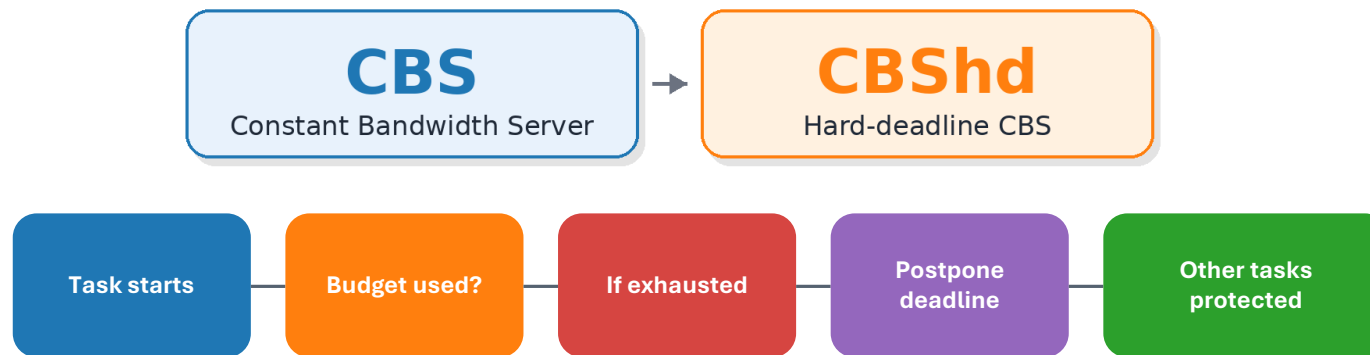
(Constant Bandwidth Server)
Reserve bandwidth and postpone deadlines when budget is exceeded.

[1][2]

Key: choose the handler according to task criticality, dependency, and acceptable performance loss.

Optimal idea: isolate the overrun, do not let it spread

CBS/CBShd intuition: budget, bandwidth, and controlled deadline postponement



Processor bandwidth

Q_s = CPU budget
 T_s = server period
 $U_s = Q_s / T_s$

In my simulation

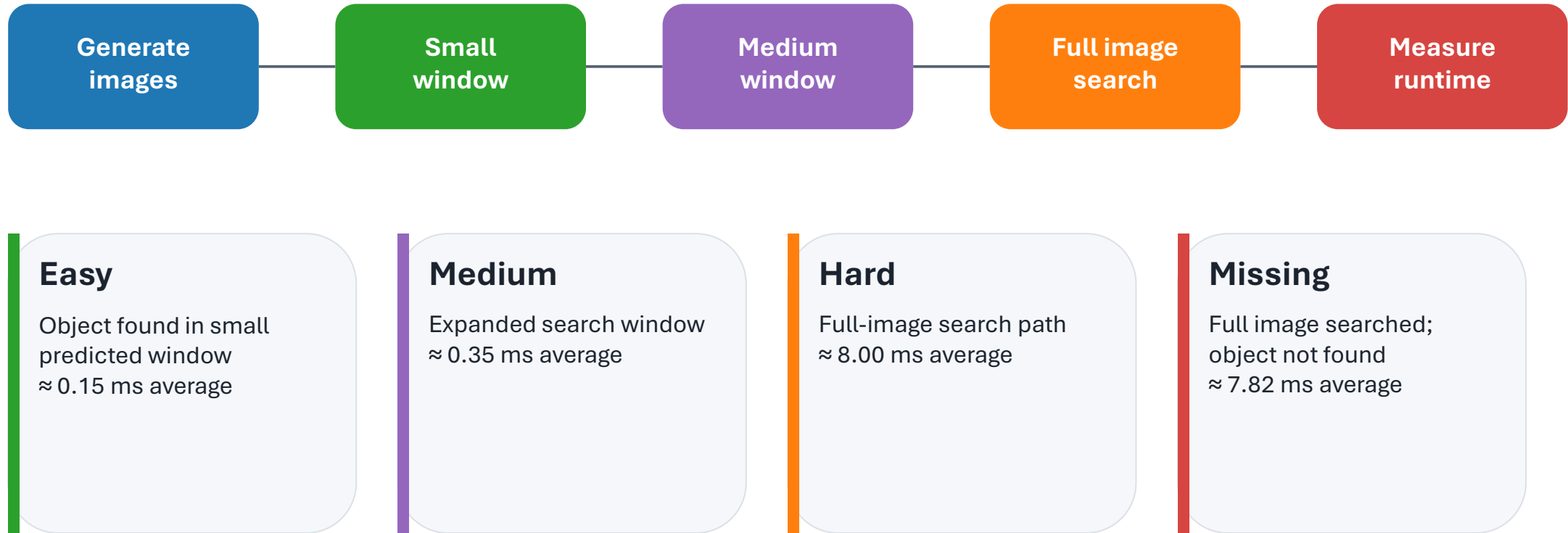
$Q_s = 0.5$ ms, $T_s = 2$ ms
 $U_s = 0.25 \rightarrow 25\%$ image-task bandwidth

The overrun is not eliminated — it is isolated and paid for by the overrunning task.

Sources: Caccamo et al. describe local overrun handling and CBShd; Abeni & Buttazzo introduce CBS resource reservation. [1][2]

What I implemented: Visual-tracking Overrun Demo

A small image-processing task creates controlled normal, hard, and worst-case paths



Important: this is a timing simulation, not a kernel-level CBS implementation.

Implementation architecture

Five scripts generate the workload, measure runtimes, and simulate handling strategies

1

generate_images.py

Creates 1920×1080 easy / medium / hard / missing frames

2

overrun_demo.py

Searches for red object and logs execution time

3

plot_results.py

Creates workload and overrun plots

4

scheduler_simulation.py

Compares WCET, NormalOnly, and OverrunHandled

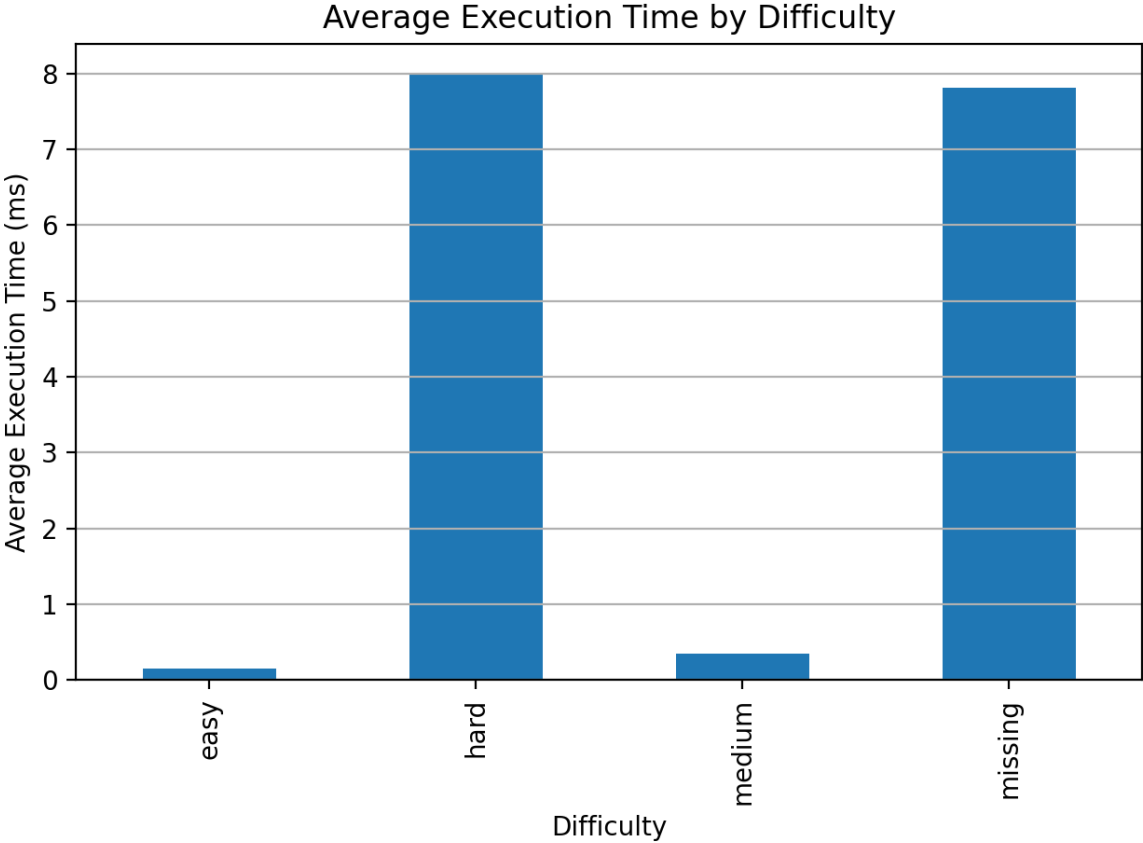
5

cpu_timeline_single_frame.py

Visualizes NormalOnly, WCET, CBS, and CBShd behavior

Measured workload: runtime depends on input difficulty

This is where the execution overruns come from in the prototype



0.15 ms

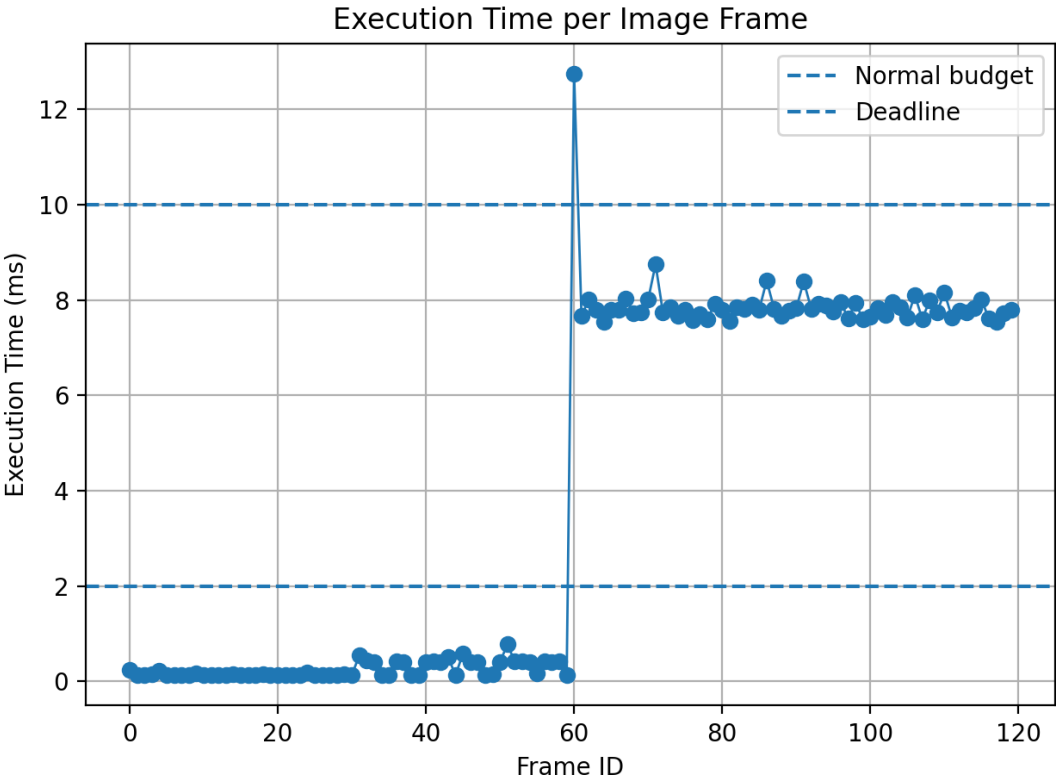
easy avg.

8.00 ms

hard avg.

7.82 ms

missing avg.



12.75 ms

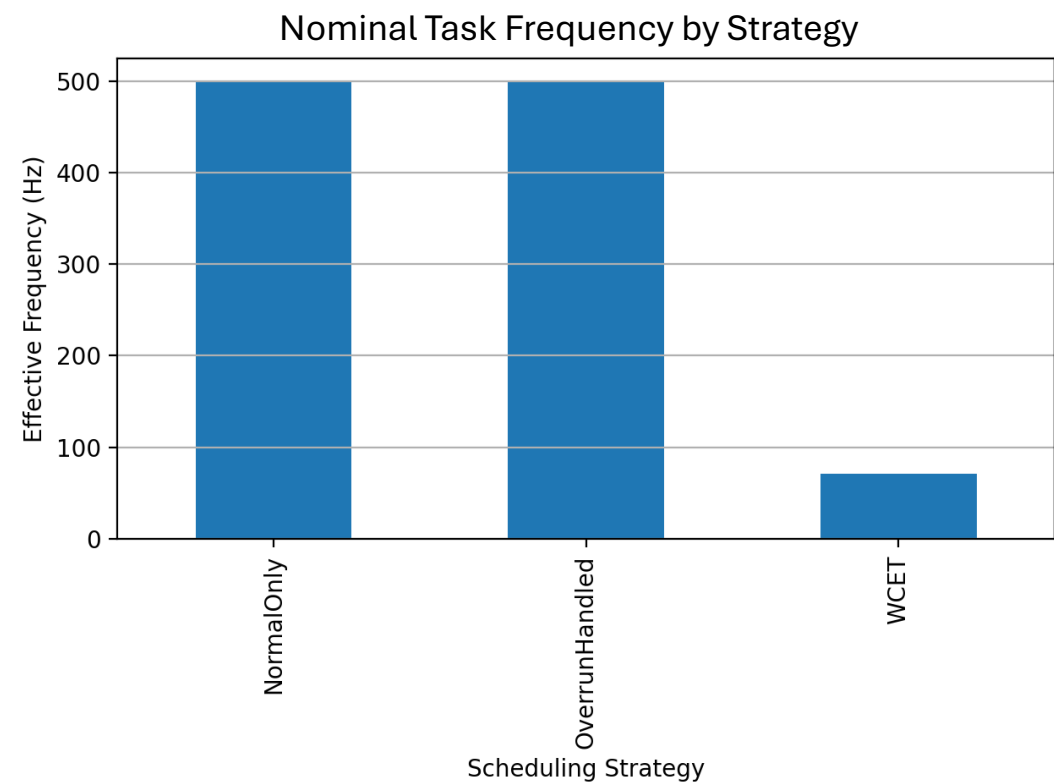
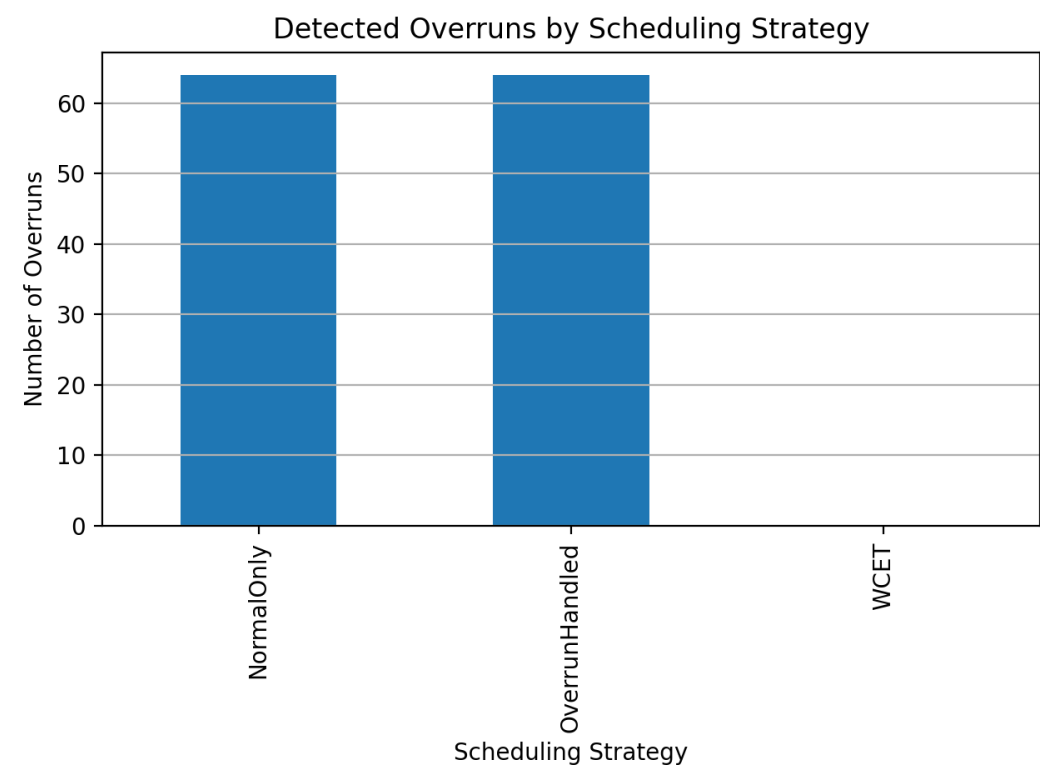
measured WCET

60

hard/missing overruns

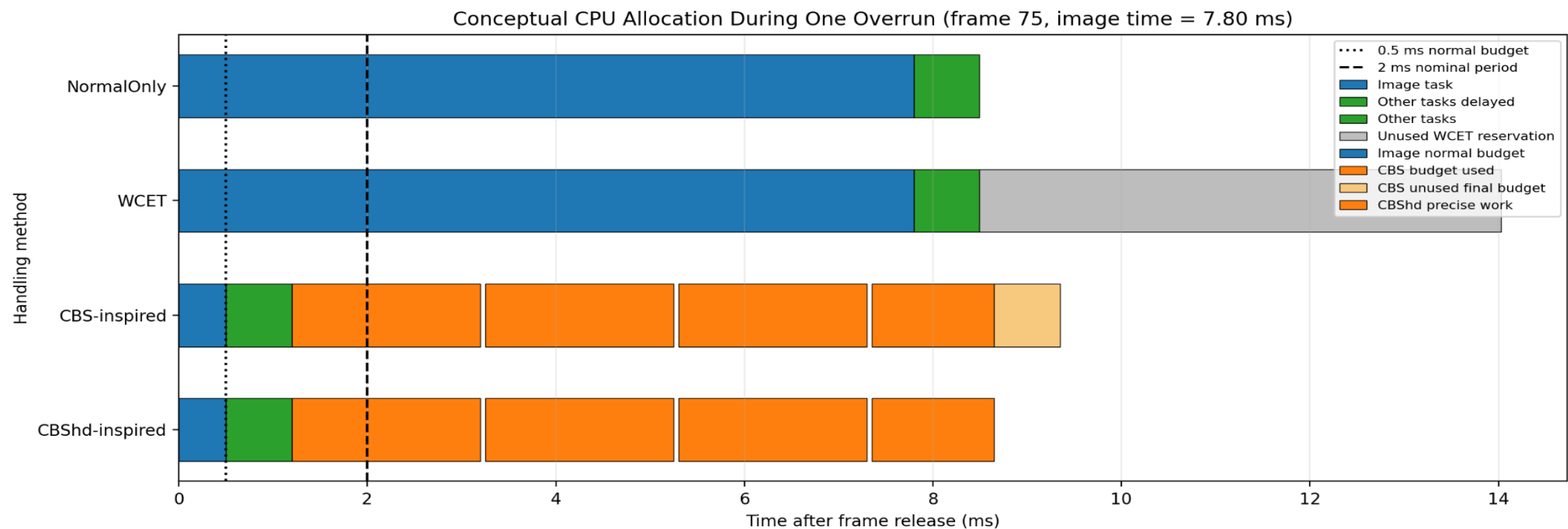
Scheduling strategy comparison

Same measured runtimes, different scheduling assumptions



Conceptual CPU allocation during one overrun

The same overrun is treated differently by each handling method



NormalOnly

Image runs to completion → other work delayed.

WCET

Safe, but reserves for worst case → unused time.

CBS

Controls overrun using budget blocks.

CBShd

Final allocation only matches remaining work.

Which handler is best used where?

Handler choice depends on criticality, dependencies, and acceptable delay

WCET-based scheduling

Best for safety-critical or sequentially dependent tasks where the next step must receive a deterministic result. Useful for actuator/control chains and hard end-to-end deadlines.

CBS / CBShd handling

Best for variable workloads such as perception, tracking, or mixed criticality tasks where occasional delay is acceptable but interference must be bounded.

Fallback / degraded output

Best when the system can safely use previous data, skip a frame, or lower computation quality to keep the control loop alive.

Important dependency rule

If Task B cannot run without Task A's result, overrun handling must be designed for the whole chain: WCET analysis, fallback output, or dedicated CPU/bandwidth may be required.

Evidence: WCET-based guarantees are the classical hard real-time approach; CBS provides resource reservation without jeopardizing guaranteed hard tasks; Caccamo et al. use CBS/CBShd to optimize control performance while maintaining schedulability. [1][2][3][4]

Limitations and connection to FreeRTOS / UPPAAL

What the prototype proves — and what it does not claim

What it proves

The same workload leads to very different deadline behavior depending on how the scheduler accounts for overruns.

What it does not claim

It is not a kernel-level CBS/CBS_{hd} implementation and does not provide hard real-time guarantees in Python.

Course connection

FreeRTOS can approximate budget checks using tasks/priorities/delays; UPPAAL can model and verify deadline-miss behavior.

Final takeaway

Overruns are not eliminated. They are isolated and accounted for so timing behavior remains controlled.

Without handling, other tasks pay the cost. With CBS/CBS_{hd}, the overrunning task pays through controlled budget/deadline behavior.

References supporting the design choices

Sources behind the claims about where each handler is useful

[1] M. Caccamo, G. Buttazzo, L. Sha, “Handling Execution Overruns in Hard Real-Time Control Systems,” IEEE Transactions on Computers, 2002. — Main source for local overrun handling, CBS_{hd}, control performance, and schedulability under worst case.

[2] L. Abeni, G. Buttazzo, “Integrating Multimedia Applications in Hard Real-Time Systems,” RTSS, 1998. — Introduces CBS resource reservation: soft/variable tasks served without jeopardizing guaranteed hard tasks.

[3] G. C. Buttazzo, Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications. — Background for WCET, hard real-time predictability, utilization, EDF, and overload handling.

[4] C. L. Liu, J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” JACM, 1973. — Foundational real-time scheduling model and schedulability concepts.

[5] T. P. Baker, “Stack-Based Scheduling of Real-Time Processes,” Real-Time Systems, 1991. — Resource sharing/SRP background for bounding blocking when tasks share resources.